UNIVERSIDAD DE LA SABANA

Internet de las Cosas

Laboratorio 2: FreeRTOS

Autores:

Angie Camila Cano Gómez

Carmen Celeste Durán Báez

Diego Eduardo Escalante García

Profesor: Fabián Páez

Abril, 2026

1. Práctica: Sistema de Lectura de Sensores con FreeRTOS

1.1 Descripción del sistema

El sistema implementado utiliza los principales elementos de FreeRTOS (tareas, colas y mutex) para gestionar la lectura de dos sensores capacitivos/touch y el envío de datos por el puerto serial en formato JSON. La arquitectura garantiza concurrencia segura mediante semáforos/mutex y reutilización de funciones mediante parámetros genéricos (void*).

1.2 Arquitectura general

El sistema se compone de cuatro tareas independientes:

- Sensor1Task y Sensor2Task: Ejecutan la misma función sensorTask() con parámetros distintos. Leen los pines T0 y T7 respectivamente, y depositan los datos junto con un timestamp en colas independientes (queue1 y queue2).
- Serial1Task y Serial2Task: Ejecutan la misma función serialTask() con parámetros distintos. Consumen los datos de cada cola y los envían al puerto serial en formato JSON, usando serialMutex para evitar acceso concurrente.

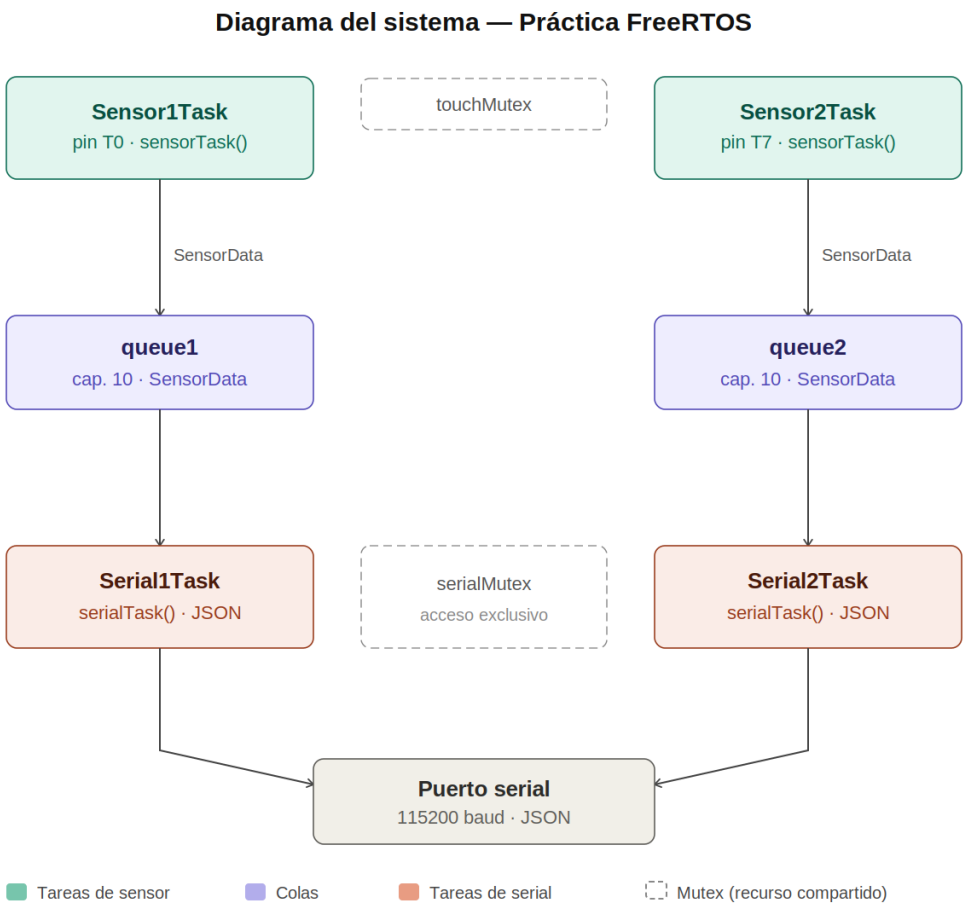

Figura 1. Diagrama del sistema: Tareas, colas y mutex implementados.

1.3 Código implementado

```
#include <Arduino.h>

typedef struct {
  int value;
  uint32_t timestamp;
} SensorData;

typedef struct {
  int pin;
  QueueHandle_t queue;
} SensorParams;

QueueHandle_t queue1;
QueueHandle_t queue2;
SemaphoreHandle_t serialMutex;
```

```
SemaphoreHandle_t touchMutex;

// Función A — lectura de sensor (reutilizada por Sensor1Task y Sensor2Task)
void sensorTask(void *pvParameters) {
  SensorParams *params = (SensorParams*) pvParameters;
  SensorData data;
  while (1) {
    if (xSemaphoreTake(touchMutex, portMAX_DELAY)) {
      data.value = touchRead(params->pin);
      xSemaphoreGive(touchMutex);
    }
    data.timestamp = millis();
    xQueueSend(params->queue, &data, portMAX_DELAY);
    vTaskDelay(pdMS_TO_TICKS(100));
  }
}

// Función B — envío JSON por serial (reutilizada por Serial1Task y Serial2Task)
void serialTask(void *pvParameters) {
  QueueHandle_t queue = (QueueHandle_t) pvParameters;
  SensorData data;
  while (1) {
    if (xQueueReceive(queue, &data, portMAX_DELAY)) {
      if (xSemaphoreTake(serialMutex, portMAX_DELAY)) {
        Serial.print("{\"value\":");
        Serial.print(data.value);
        Serial.print(",\"time\":");
        Serial.print(data.timestamp);
        Serial.println("}");
        xSemaphoreGive(serialMutex);
      }
    }
  }
}

void setup() {
  Serial.begin(115200);
  queue1 = xQueueCreate(10, sizeof(SensorData));
  queue2 = xQueueCreate(10, sizeof(SensorData));
  serialMutex = xSemaphoreCreateMutex();
  touchMutex  = xSemaphoreCreateMutex();
  static SensorParams sensor1Params = { T0, queue1 };
  static SensorParams sensor2Params = { T7, queue2 };
  touchAttachInterrupt(T0, NULL, 40);
  touchAttachInterrupt(T7, NULL, 40);
  xTaskCreate(sensorTask, "Sensor1", 4096, &sensor1Params, 1, NULL);
  xTaskCreate(sensorTask, "Sensor2", 4096, &sensor2Params, 1, NULL);
  xTaskCreate(serialTask, "Serial1", 4096, queue1, 1, NULL);
  xTaskCreate(serialTask, "Serial2", 4096, queue2, 1, NULL);
}

void loop() { /* vacío — FreeRTOS gestiona el scheduler */ }
```

2. Preguntas Conceptuales

2.1 ¿Cómo se ejecutan tareas de FreeRTOS con la misma función, pero distintos parámetros?

En FreeRTOS, la función xTaskCreate() permite crear múltiples tareas que apuntan a la misma función de código, diferenciándolas únicamente por los argumentos que se les pasa como cuarto parámetro (pvParameters). FreeRTOS asigna un stack independiente de memoria a cada instancia creada, de modo que, aunque dos tareas compartan el mismo código de función, cada una posee su propio contexto de ejecución, variables locales y flujo independiente.

En el código implementado, sensorTask se registra dos veces: Una con sensor1Params (pin T0 y queue1) y otra con sensor2Params (pin T7 y queue2). La misma lógica aplica para serialTask. Así se logra la reutilización de funciones exigida por la práctica.

2.2 ¿Cuál es el tipo de dato que recibe una tarea de FreeRTOS? ¿Cómo se convierte al tipo específico requerido?

Toda función de tarea recibe un único parámetro de tipo void*, que es un puntero genérico capaz de apuntar a cualquier tipo de dato en memoria. Esta decisión de diseño permite que la interfaz de xTaskCreate() sea completamente genérica, independientemente de qué tipo de parámetro necesite cada tarea.

La conversión al tipo específico se realiza mediante un cast explícito al inicio de la función. En el código hay dos conversiones distintas: En sensorTask se castea a SensorParams* para acceder a params->pin y params->queue; en serialTask se castea directamente a QueueHandle_t, ya que ese tipo es internamente un puntero y cabe en un void*. Es importante que sensor1Params y sensor2Params se declaren como static dentro de setup(), garantizando que los structs permanezcan en memoria mientras las tareas estén activas.

2.3 ¿Qué pasa cuando una cola se llena y una tarea quiere insertar nuevos elementos?

Las colas se crean con una capacidad fija (10 elementos de tipo SensorData). Cuando la cola está llena y una tarea intenta insertar un nuevo elemento, el comportamiento depende del parámetro xTicksToWait de xQueueSend():

- Con portMAX_DELAY (usado en el código): La tarea productora se bloquea indefinidamente hasta que haya espacio disponible. El scheduler cede el CPU a otras tareas, sin consumo inútil de procesador.
- Con timeout de cero: xQueueSend() retorna inmediatamente con errQUEUE_FULL y el dato se pierde.

- Con timeout finito: La tarea espera hasta ese tiempo máximo y, si sigue llena, retorna errQUEUE_FULL.

En la práctica, el uso de portMAX_DELAY es apropiado cuando la pérdida de datos no es aceptable.

2.4 ¿Es posible que varias tareas lean y escriban a la misma cola?

Sí ya que las colas de FreeRTOS son thread-safe por diseño, internamente utilizan secciones críticas para proteger las operaciones de inserción y extracción, garantizando que no ocurra corrupción de datos aunque múltiples tareas accedan concurrentemente. Cuando múltiples consumidores leen de la misma cola, cada elemento es entregado exactamente a una sola tarea, la primera que llame xQueueReceive() cuando haya datos disponibles, sin duplicación. No es necesario un mutex adicional para proteger la cola, a diferencia del puerto serial que sí requiere serialMutex porque las llamadas a Serial.print() no son atómicas.

2.5 ¿Qué es un deadlock? Ejemplo de código y explicación

Un deadlock (interbloqueo) ocurre cuando dos o más tareas se bloquean mutuamente esperando un recurso que la otra ya retiene, formando un ciclo del que ninguna puede salir. Las cuatro condiciones necesarias son: Exclusión mutua, retención y espera, no apropiación, y espera circular.

En el sistema implementado, si sensorTask toma touchMutex y luego intenta tomar serialMutex, mientras serialTask toma serialMutex y luego intenta tomar touchMutex, se crea una espera circular cuando el scheduler ejecuta ambas casi simultáneamente.

```
// Código que GENERA DEADLOCK — orden opuesto de adquisición
void sensorTask(void *pvParameters) {
  while (1) {
    xSemaphoreTake(touchMutex, portMAX_DELAY);   // toma touch primero
    xSemaphoreTake(serialMutex, portMAX_DELAY);  // luego serial → DEADLOCK
    Serial.println("Leyendo sensor...");
    xSemaphoreGive(serialMutex);
    xSemaphoreGive(touchMutex);
    vTaskDelay(pdMS_TO_TICKS(100));
  }
}
void serialTask(void *pvParameters) {
  while (1) {
    xSemaphoreTake(serialMutex, portMAX_DELAY);  // toma serial primero
    xSemaphoreTake(touchMutex, portMAX_DELAY);   // luego touch → DEADLOCK
    Serial.println("Enviando dato...");
    xSemaphoreGive(touchMutex);
    xSemaphoreGive(serialMutex);
    vTaskDelay(pdMS_TO_TICKS(100));
  }
}
```

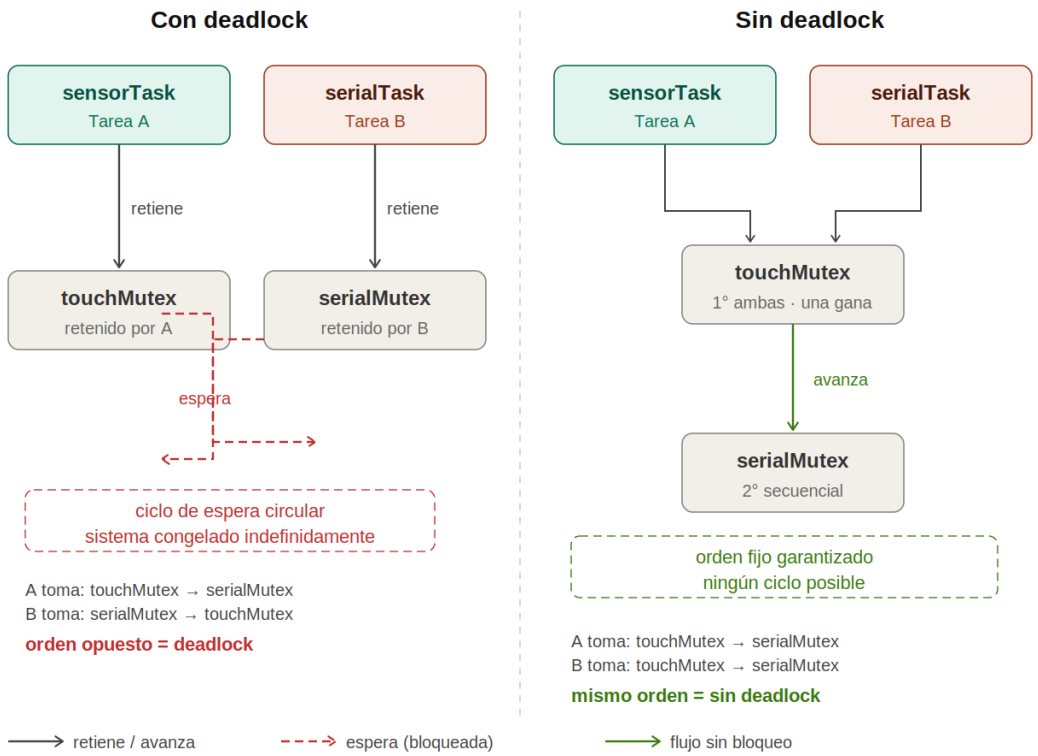

Figura 2. Izquierda: Ciclo de espera circular que genera deadlock. Derecha: Solución con orden fijo.

La solución más efectiva es imponer un orden fijo de adquisición en todas las tareas. Si siempre se toma primero touchMutex y luego serialMutex, nunca puede formarse el ciclo:

```
// Código SIN DEADLOCK — mismo orden en ambas tareas
void sensorTask(void *pvParameters) {
  while (1) {
    xSemaphoreTake(touchMutex, portMAX_DELAY);   // siempre touch primero
    xSemaphoreTake(serialMutex, portMAX_DELAY);  // luego serial
    Serial.println("Leyendo sensor...");
    xSemaphoreGive(serialMutex);
    xSemaphoreGive(touchMutex);
    vTaskDelay(pdMS_TO_TICKS(100));
  }
}
void serialTask(void *pvParameters) {
  while (1) {
    xSemaphoreTake(touchMutex, portMAX_DELAY);   // mismo orden
    xSemaphoreTake(serialMutex, portMAX_DELAY);
    Serial.println("Enviando dato...");
    xSemaphoreGive(serialMutex);
    xSemaphoreGive(touchMutex);
    vTaskDelay(pdMS_TO_TICKS(100));
  }
```

```
}
```

2.6 Ejemplo de datos inconsistentes sin mutex vs. con mutex

Sin serialMutex, las dos instancias de serialTask pueden interrumpirse mutuamente en medio de una transmisión serial, ya que cada llamada a Serial.print() es una operación separada y el scheduler puede preemptar la tarea entre cualquiera de ellas:

```
// SIN serialMutex — genera datos inconsistentes
void serialTask(void *pvParameters) {
  QueueHandle_t queue = (QueueHandle_t) pvParameters;
  SensorData data;
  while (1) {
    if (xQueueReceive(queue, &data, portMAX_DELAY)) {
      Serial.print("{\"value\":");   // Sensor1 imprime hasta aquí
      // scheduler cambia a Sensor2, que imprime su JSON completo
      Serial.print(data.value);       // Sensor1 continúa mezclado
      Serial.print(",\"time\":");
      Serial.print(data.timestamp);
      Serial.println("}");
    }
  }
}
// Salida observable (JSON inválido):
// {"value":23{"value":45,"time":1002}
// ,"time":998}
```

```
// CON serialMutex — datos consistentes
void serialTask(void *pvParameters) {
  QueueHandle_t queue = (QueueHandle_t) pvParameters;
  SensorData data;
  while (1) {
    if (xQueueReceive(queue, &data, portMAX_DELAY)) {
      if (xSemaphoreTake(serialMutex, portMAX_DELAY)) {
        // solo una tarea a la vez entra aquí
        Serial.print("{\"value\":");
        Serial.print(data.value);
        Serial.print(",\"time\":");
        Serial.print(data.timestamp);
        Serial.println("}");
        xSemaphoreGive(serialMutex);
        // Salida limpia garantizada:
        // {"value":23,"time":998}
        // {"value":45,"time":1002}
      }
    }
  }
}
```

Mientras una tarea retiene serialMutex y ejecuta todas las llamadas a Serial.print(), la otra queda bloqueada en xSemaphoreTake() y no puede acceder al puerto serial hasta que el bloque completo termine y el mutex sea liberado, garantizando siempre JSON válido.

3. Arquitectura del Proyecto Final

3.1 Descripción general

A continuación, se plantea una arquitectura basada en tareas, colas y semáforos/mutex para HydroSense, un sistema IoT de monitoreo de humedad del suelo y riego automático basado en ESP32. La arquitectura separa responsabilidades en tareas independientes que se comunican mediante colas y controlan el acceso a recursos compartidos con mutex.

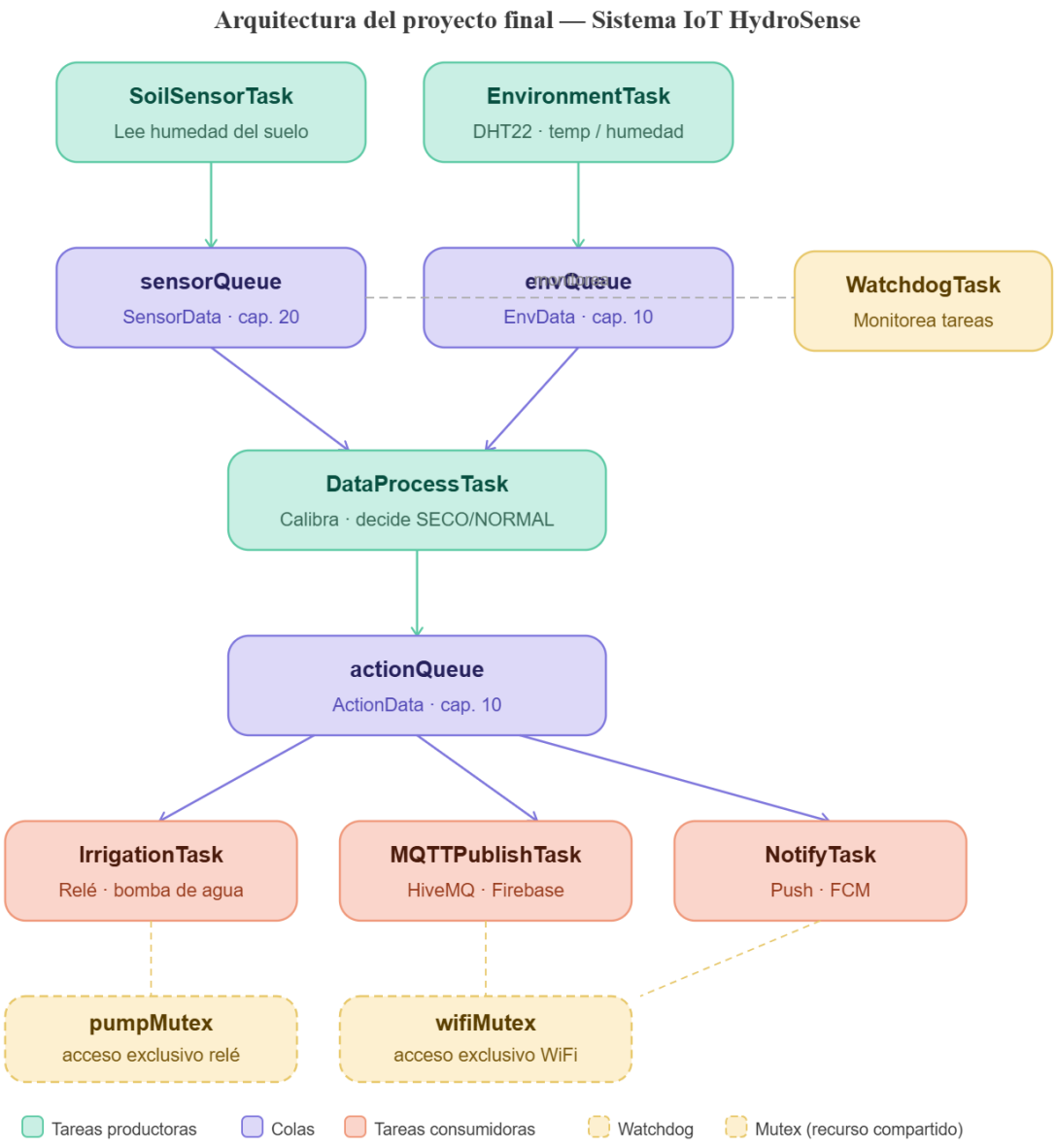

Figura 3. Diagrama de componentes de la arquitectura hipotética del proyecto final.

3.2 Tareas identificadas

El sistema se descompone en cinco tareas con responsabilidades bien delimitadas:

- SoilSensorTask: Lee periódicamente los dos sensores de humedad del suelo (ADC1, GPIO32–GPIO39) y deposita las lecturas en sensorQueue. Promedia 20 muestras para reducir ruido del ADC.
- DataProcessTask: Extrae datos de sensorQueue, aplica la lógica de decisión (SECO < 30%, SATURADO > 75%) y deposita la acción correspondiente en actionQueue.
- IrrigationTask: Consume actionQueue y activa o desactiva el relé que controla la bomba de agua, protegida por pumpMutex.
- MQTTPublishTask: Consume actionQueue y publica los datos al broker HiveMQ Cloud vía MQTT para su almacenamiento en Firebase, protegida por wifiMutex.
- NotifyTask: Evalúa actionQueue y, cuando detecta estado SECO crítico, solicita el envío de notificación push al usuario vía Firebase Cloud Messaging, también protegida por wifiMutex.

3.3 Colas identificadas

- sensorQueue: Transfiere SensorData (valor ADC promediado + timestamp + ID de sensor) desde SoilSensorTask hacia DataProcessTask. Capacidad sugerida: 20 elementos.
- actionQueue: Transfiere ActionData (decisión: regar/notificar/idle + nivel de humedad calculado) desde DataProcessTask hacia IrrigationTask, MQTTPublishTask y NotifyTask. Capacidad sugerida: 10 elementos.

3.4 Recursos compartidos y control de concurrencia

- wifiMutex: protege el módulo WiFi/MQTT del ESP32 de acceso simultáneo entre MQTTPublishTask y NotifyTask, ya que ambas requieren la conexión de red.
- pumpMutex: protege el relé de la bomba de agua para evitar que IrrigationTask y una eventual tarea de control manual desde el dashboard activen la bomba simultáneamente.

La arquitectura cumple con los tres principios fundamentales de FreeRTOS: Separación de responsabilidades en tareas independientes, comunicación asíncrona mediante colas thread-safe, y control de acceso a recursos compartidos mediante mutex. Esto permite escalar el sistema fácilmente, por ejemplo, añadiendo sensores de temperatura o una tarea de deep sleep sin modificar las tareas existentes.

4. Referencias

Barry, R. (2023). Mastering the FreeRTOS real time kernel: A hands-on tutorial guide. Real Time Engineers Ltd. https://www.freertos.org/Documentation/RTOS_book.html

Espressif Systems. (2024). ESP-IDF FreeRTOS SMP changes. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/freertos-smp.html

FreeRTOS. (2024). FreeRTOS API reference. https://www.freertos.org/a00106.html

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). John Wiley & Sons.

Tanenbaum, A. S., & Bos, H. (2014). Modern operating systems (4th ed.). Pearson Education.